P3301R0
inplace_stoppable_base

Published Proposal, 2024-05-21

Author:
Lauri Vasama

Audience:
LEWG

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Current draft:
vasama.github.io/wg21/D3301.html

Current draft source:
github.com/vasama/wg21/blob/main/D3301.bs

Abstract

Introduce a CRTP base class template as a more efficient alternative to inplace_stop_callback.

Table of Contents

§ 1. Introduction

Any sender [P2300R9] wishing to support cancellation using cancellation tokens, particularly `std::inplace_stop_token`, must currently store as a subobject of its operation state a `std::inplace_stop_callback` object containing a reference to the operation state itself:

```
class my_sender {
  template<typename R>
  class op_state {
    R receiver;

    class cb_t {
      op_state& op;
    public:
      cb_t(op_state& op)
        : op(op) {}
      void operator()() const {
        op.on_stop_requested();
      }
    };
    std::inplace_stop_callback<cb_t> cb;

    void on_stop_requested();
  };
};
```

This naturally comes with a small but non-zero cost in the space required for each operation state. While standard library implementations could likely eliminate this cost for any standard senders in just the manner described in this paper, that will not be possible for any user senders wishing to make use of `std::inplace_stop_token` as it is currently specified.

§ 2. Proposal

We propose to introduce a new standard library CRTP class template for use as a base subobject of classes which would otherwise contain as a subobject a `std::inplace_stop_callback` with a reference to the enclosing object:

```
template<typename T>
class inplace_stoppable_base;
```

As is usual with CRTP, it is required that the class type `T` is derived (directly or indirectly) from `inplace_stoppable_base<T>` and that the conversion from `inplace_stoppable_base<T>*` to `T*` is accessible to `inplace_stoppable_base<T>`.

Unlike `inplace_stop_callback`, the only constructor of `inplace_stoppable_base` has `protected` access and takes a single parameter of type `inplace_stop_token`.

Upon request for cancellation of the `inplace_stop_token` to which a `inplace_stoppable_base` object is registered, the member function `on_stop_requested()` is invoked on the derived object of type `T`.

Using this class template, the previous example can not only be optimised, but vastly simplified:

```
class my_sender {
  template<typename R>
  class op_state : std::inplace_stoppable_base<op_state<R>> {
    R receiver;
    // invoked upon request for cancellation.
    void on_stop_requested();
  };
};
```

Note that it is possible to implement `std::inplace_stop_callback` in terms of `inplace_stoppable_base`, while the reverse is not true.

§ 2.1. Naming of the derived member function

We propose the name `on_stop_requested` which has much precedent in libraries in the wild and in other languages, but little directly applicable precedent in the standard library. The standard library does include a few things named similarly:

- Allocator propagation traits such as `propagate_on_container_move_assignment`.
- A few functions related to `iostream` such as `set_emit_on_sync`.

Other possible names include:

1. `stop_requested`
   This is a reasonable name, but `inplace_stop_token` itself has a function with the same name used for a different but related purpose.
2. `at_stop_requested`
   This pattern has existing precedent in the standard library for callback functions:
   - `atexit`
   - `at_quick_exit`.

Note that it is fairly trivial for users to create a thin wrapper which uses their desired naming:

```
template<typename T>
class InplaceStoppableBase : std::inplace_stoppable_base<InplaceStoppableBa
  friend std::inplace_stoppable_base<InplaceStoppableBase<T>>;
protected:
  using std::inplace_stoppable_base<InplaceStoppableBase<T>>::inplace_stopp
private:
  void on_stop_requested() noexcept {
    static_cast<T*>(this)->OnStopRequested();
  }
};
```

§ 2.2. Interaction with `stop_callback_for_t`

This proposal does not include any changes to `stop_callback_for_t` or the stop token concepts.

It is possible for users to handle arbitrary stop token types by specializing for types known to support CRTP and falling back to the callback type otherwise:

```
template<typename T, typename Token>
class my_stoppable_base_for {
  class cb_t {
    my_stoppable_base_for& base;
```

```
  public:
    explicit cb_t(my_stoppable_base_for& base)
      : base(base) {}
    void operator()() {
      static_cast<T&>(base).on_stop_requested();
    }
  };
  std::stop_callback_for_t<Token, cb_t> cb;

protected:
  explicit my_stoppable_base_for(Token token)
    : cb(token) {}
};

template<typename T>
class my_stoppable_base_for<T, std::inplace_stop_token>
  : public std::inplace_stoppable_base<T> {
  friend std::inplace_stoppable_base<T>;
protected:
  using std::inplace_stoppable_base::inplace_stoppable_base;
};
```

§ 3. Alternatives considered

§ 3.1. `inplace_stop_callback` optional parameter

Instead of introducing a new class template, `inplace_stop_callback` could be modified such that a reference to the `inplace_stop_callback<CB>` itself is passed in the invocation of its *stop-callback* member, if such an invocation is viable. In addition to no new standard library template, this avoids choosing a name to be imposed on the user operation state.

We prefer the proposed `inplace_stoppable_base` over this approach mainly due to the increased complexity of user code in the common case:

```
class my_sender {
  template<OpState>
  struct cb_t {
    static void operator()(std::inplace_stop_callback<cb_t<OpState>>& cb) {
      static_cast<OpState&>(cb).on_stop_requested();
    }
```

```
  };

  template<typename R>
  class op_state : std::inplace_stop_callback<cb_t<op_state<R>>> {
    friend my_sender;
    R receiver;
    void on_stop_requested();
  };
```

Note that both this modified `inplace_stop_callback` (under another user-chosen name) and `inplace_stoppable_base` are implementable in terms of the other.

§ 3.2. Superobject accessor function

A new standard library function template `std::get_superobject` which, given a reference to the *stop-callback* object, returns a reference to the enclosing `std::inplace_stop_callback<CB>` object:

```
class my_sender {
  template<OpState>
  struct cb_t {
    void operator()() {
      using super_type = std::inplace_stop_callback<cb_t<OpState>>;
      static_cast<OpState&>(std::get_superobject<super_type&>(*this)).on_s
    }
  };

  template<typename R>
  class op_state : std::inplace_stop_callback<cb_t<op_state>> {
    friend my_sender;
    R receiver;
    void on_stop_requested();
  };
};
```

The other alternatives are implementable in terms of this option, while the reverse is not true. In that sense this the most general solution. However, apart from introducing this new function template and

all the bikeshedding that comes with that, there is one major downside: In order to derive a reference to the `inplace_stop_callback` from a reference to its *stop-callback* member, one of three conditions must be fulfilled:

1. The *stop-callback* must be a base subobject of the `inplace_stop_callback`, which in turn requires the user callback type to be non-final.
2. The `inplace_stop_callback` must be a standard layout type, which in turn requires the user callback type itself to also be a standard layout type.
3. The standard library must use compiler magic not accessible to the user in the implementation of `inplace_stop_callback`.

§ 4. Implementation experience

This proposal has been implemented at github.com/vasama/stdexec.

§ References

§ Normative References

[P2300R9]
Eric Niebler, Michał Dominiak, Georgy Evtushenko, Lewis Baker, Lucian Radu Teodorescu, Lee Howes, Kirk Shoop, Michael Garland, Bryce Adelstein Lelbach. *`std::execution`*. 2 April 2024. URL: https://wg21.link/p2300r9